

Übungsblatt 7: Tracing & Timing-Analyse

Musterlösung · Echtzeitsysteme (EZS)

Prof. Dr.-Ing. Daniel Klünder

Musterlösung

Zielsetzung

In dieser Übung verlassen Sie die reine Programmierung und wechseln in die Rolle des System-Analysten. Sie analysieren das fehlerhafte Multicore-System aus Termin 6 (den *Cache Coherency Failure*) mit professioneller Hardware-Trace-Technologie. Mithilfe der **TASKING BlueBox** und der Analysesoftware **winIDEA** lernen Sie, wie man ein System rückwirkungsfrei (*non-intrusive*) beobachtet, um die Grenzen des klassischen Debuggings (Beobachter-Effekt und *Heisenbugs*) zu überwinden.

Aufgabe 1: Hardware-Setup und Workspace

Im ersten Schritt bereiten wir die Hardware und die Analyse-Software vor.

1. Schalten Sie das Board aus und stecken Sie das 20-Pin-Flachbandkabel der BlueBox vorsichtig auf den Trace-Port des Boards. Schalten Sie anschließend beides ein.
2. Starten Sie die Software winIDEA und erstellen Sie einen neuen Workspace.
3. Laden Sie die **.elf**-Dateien beider Kerne (CM7 und CM4) in den Workspace, da diese die DWARF-Debug-Symbole enthalten, die Speicheradressen auf C-Code-Zeilen mappen.
4. Konfigurieren Sie die Taktfrequenz der CPU in winIDEA zwingend auf 400 MHz.

Frage: Warum ist die exakte Konfiguration der CPU-Taktfrequenz (400 MHz) in winIDEA so essenziell für die Timing-Analyse?

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

Musterlösung Aufgabe 1

Die Trace-Hardware (ETM/DWT) funkt nur rohe Hardware-Ereignisse und CPU-Taktzyklen nach außen an die BlueBox. Das Analysetool winIDEA weiß ohne diese Konfiguration nicht, wie lange ein einzelner Taktzyklus physikalisch dauert. Ist die Taktfrequenz falsch eingestellt, kann die Software die gemessenen Taktzyklen nicht in korrekte absolute Zeitstempel (Mikrosekunden oder Millisekunden) umrechnen, wodurch alle abgelesenen Latenzzeiten und Zeitachsen im Profiler falsch wären.

Aufgabe 2: OS-Awareness (FreeRTOS)

Der Hardware-Tracer zeichnet primär C-Funktionen und Speicheradressen auf. Um ein klares Gantt-Chart unserer FreeRTOS-Tasks zu erhalten, müssen wir dem Tool beibringen, das Betriebssystem zu verstehen.

1. Gehen Sie in winIDEA auf *Plugins* und aktivieren Sie *FreeRTOS*.

Frage: Erklären Sie das Konzept der OS-Awareness. Wie schafft es die BlueBox, völlig rückwirkungsfrei (ohne Code-Overhead auf der CPU) herauszufinden, welche FreeRTOS-Task gerade den Zustand *Running* besitzt?

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

Musterlösung Aufgabe 2

Die OS-Awareness nutzt eine globale System-Variable in FreeRTOS namens `pxCurrentTCB`. Diese Variable zeigt in jedem Moment als Pointer auf den *Task Control Block* der Task, die gerade die CPU besitzt.

winIDEA sucht in der geladenen `.elf`-Datei nach der physikalischen Speicheradresse von `pxCurrentTCB`. Anschließend programmiert die Software das DWT-Hardwaremodul (*Data Watchpoint and Trace*) im ARM-Chip so, dass es exakt diese RAM-Adresse beobachtet. Jedes Mal, wenn der FreeRTOS-Scheduler (PendSV) den Pointer ändert (Taskwechsel), erkennt das DWT dies auf dem Bus und funkt die neue Adresse sofort an die BlueBox. winIDEA gleicht den Pointer mit der Task-Liste ab und zeichnet daraus live das Gantt-Chart (*Running*-Balken).

Aufgabe 3: Profiler und DWT-Tracing einrichten

Wir wollen nun die fehlerhafte Variable aus dem Shared Memory der 6. Laborübung überwachen. In Übungsblatt 6 heißt die Shared-Variable konsistent `sd` (Typ `SharedData_t`); in den Vorlesungsfolien erscheint dasselbe Struct teils noch als `ping_pong_data`. Beobachten Sie das Flag `sd.m7_ready`.

1. Öffnen Sie in winIDEA den *Analyzer / Profiler*.
2. Fügen Sie einen DWT-Data-Trace für die C-Variable `sd.m7_ready` hinzu.

Frage: Warum nutzen wir für die Überwachung dieses Flags das DWT-Modul von ARM CoreSight und loggen den Variablen-Zustand nicht einfach durch ein `printf("Flag ist jetzt %d", sd.m7_ready);` im C-Code des Cortex-M7?

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

Musterlösung Aufgabe 3

Der Aufruf von `printf()` blockiert die Ausführung der CPU massiv. Das Senden weniger Buchstaben (z. B. „Hallo“, 5 Bytes) über einen UART-Port bei Standard 115200 Baud dauert bei Rahmenformat **8N1** (10 Bits/Byte) bereits

$$5 \times 10 \times \frac{1}{115200} \text{ s} \approx 434 \text{ s}$$

(in der Vorlesung gerundet ≈ 430 s). Das Einfügen eines solchen Befehls in harten Echtzeit-Code verschiebt das Timing extrem und kann sogenannte *Heisenbugs* erzeugen, bei denen sich Race Conditions durch den Beobachtungsversuch zufällig verändern oder verschwinden.

Das DWT-Modul hingegen überwacht den RAM-Speicherbus physikalisch parallel zur CPU. Es protokolliert die Variablenänderungen zu 100 % rückwirkungsfrei und erzeugt dabei exakt 0 Taktzyklen Overhead für den Prozessor.

Aufgabe 4: Aufzeichnung und Analyse des Cache-Fehlers

Wir führen nun die eigentliche Messung durch und beweisen den Fehler auf der Hardware.

1. Nutzen Sie das kaputte Ping-Pong-Projekt aus Termin 6, bei dem die MPU (*Memory Protection Unit*) im Code mutwillig **deaktiviert** wurde.
2. Starten Sie das Programm auf dem STM32H755. Drücken Sie in winIDEA auf *Record* (den roten Punkt) und lassen Sie die Aufnahme für wenige Sekunden laufen.
3. Beenden Sie die Aufnahme und betrachten Sie die ausgewertete Timeline im Analyzer.

Frage: Beschreiben Sie das Aha-Erlebnis im Analyzer: Was sehen Sie in den Graphen für die M4-Task, für den M7-Kern und für die DWT-Daten-Variable `sd.m7_ready`? Welches physikalische Architektur-Problem haben Sie damit live bewiesen?

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

Musterlösung Aufgabe 4

Beobachtungen in der Timeline:

1. Man sieht, dass die M4-Task in ihrem `osDelay()`-Aufruf feststeckt und quasi „verhungert“.
2. Der C-Code des Cortex-M7 rechnet jedoch munter weiter.
3. Der Daten-Graph des DWT für die Variable `sd.m7_ready` bleibt eine absolut flache Nulllinie; er ändert sich im echten SRAM niemals.

Bewiesenes Problem:

Diese Analyse ist der visuelle Beweis für den **Cache Coherency Failure**. Obwohl der C-Code auf dem Cortex-M7 den Befehl `sd.m7_ready = 1` erfolgreich ausführt, sorgt das Write-Back-Verfahren dafür, dass diese 1 ausschließlich im internen L1-Cache des M7 landet. Da der Cache nicht durch die MPU deaktiviert wurde, werden die Daten nicht in den physikalischen RAM durchgeschrieben. Der DWT lauscht jedoch auf dem echten SRAM-Bus und sieht dort weiterhin nur die alte 0. Da der M4 (ohne Data-Cache) denselben physikalischen Bus abfragt und dort weiterhin 0 liest, wartet er endlos in seiner `while (sd.m7_ready == 0)`-Schleife mit `osDelay(1)`.